#10

The Register File and Building an ALU

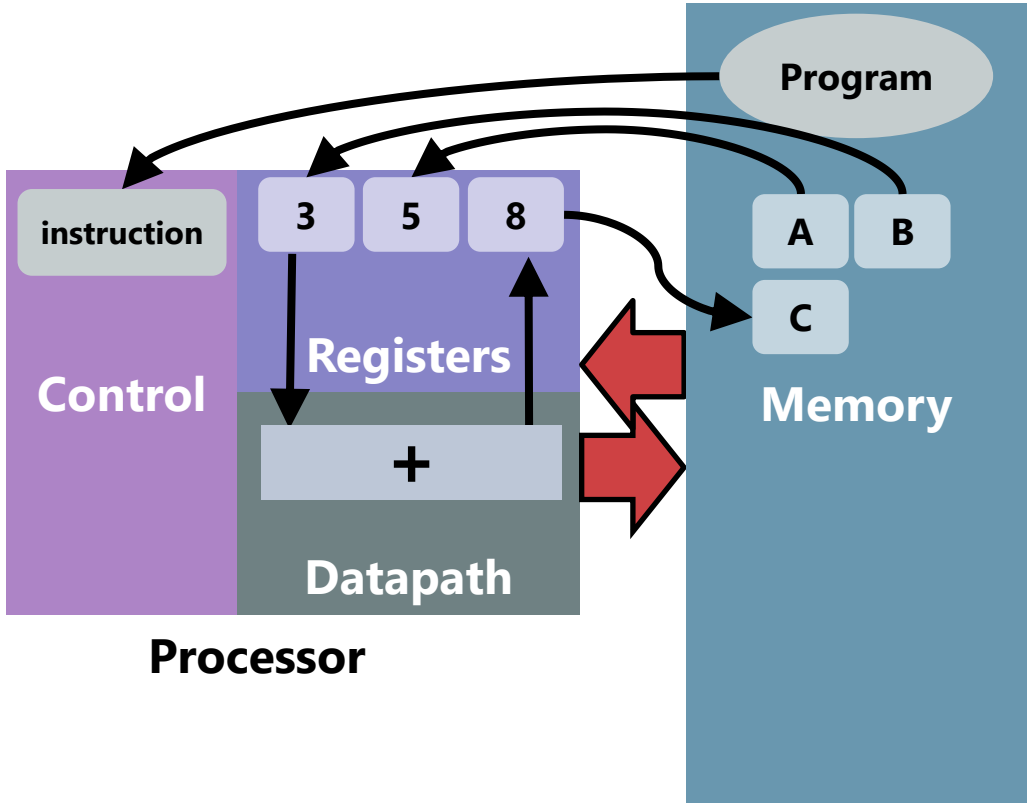
CS 0447
Introduction to
Computer Programming

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

The Register File

Remember this?



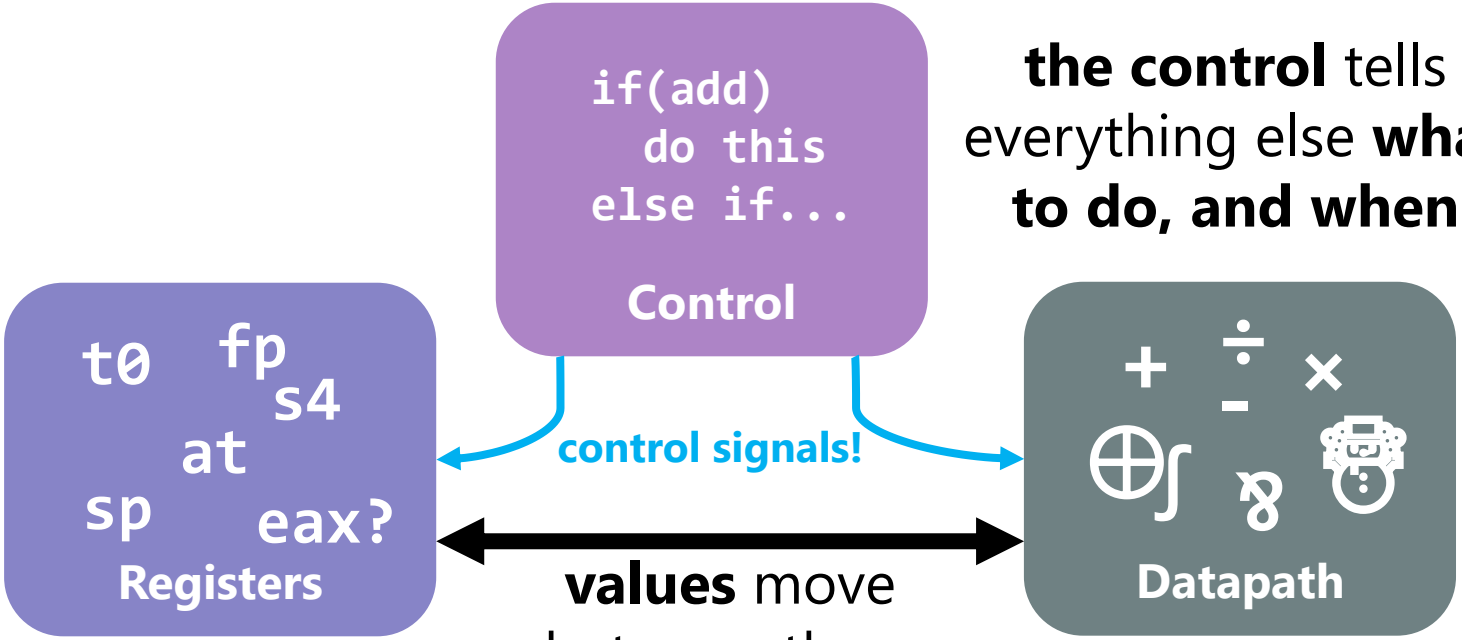
Unresolved questions:

- What's the **control?**
- What's the **datapath?**
- How does it know what instruction to get next?
- How does it know what registers to access?
- How does it know to add, subtract, etc.?

Zooming in

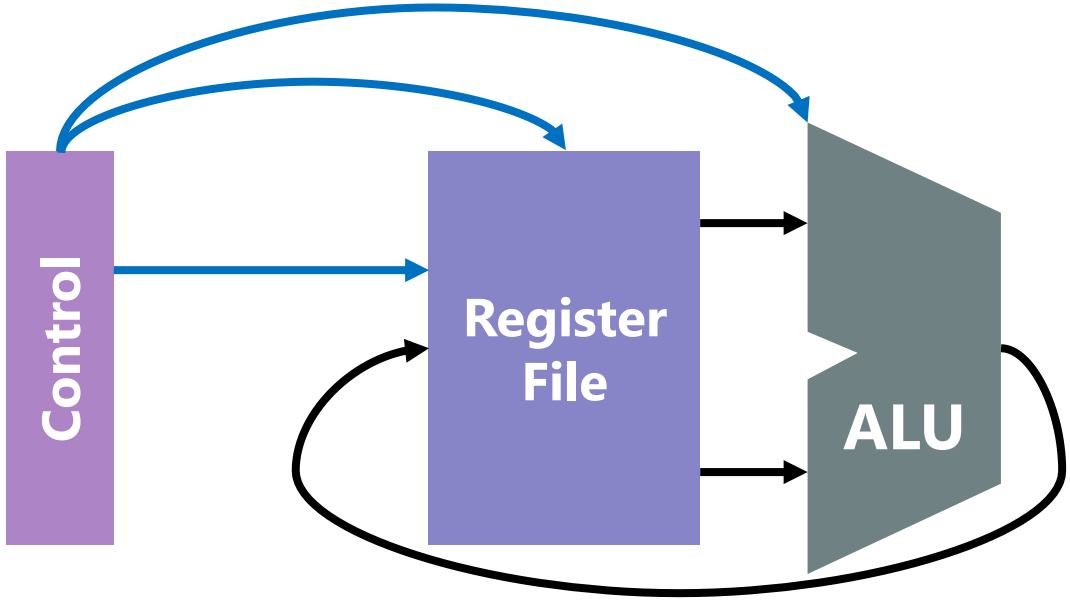
- There are a few major parts of any CPU:

...oh yeah, memory too



A bit less abstract

- The registers are grouped together into the **register file**
- The **ALU** (arithmetic and logic unit) is the main part of the datapath
- The control is doing its thing, somehow…? (we'll see)



we can get the values of **two registers at once**

It doesn't have to be this way

CISC CPUs usually have small sets of registers, and many have special purposes or behaviors

8086	z80		6502	PDP8
ax	a	*f*	A	AC
bx	b	c	X	
cx	d	e	Y	
dx	h	l		
si	ix			
di	iy			
sp	sp			
bp				

12 bits

8 bits

16 bits

RISC CPUs usually have 32* mostly-interchangeable registers: MIPS, RISC, SPARC, ARMv8, RISC-V...

r0	r1	r2	r3	r4	r5	r6	r7
r8	r9	r10	r11	r12	r13	r14	r15
r16	r17	r18	r19	r20	r21	r22	r23
r24	r25	r26	r27	r28	r29	r30	r31

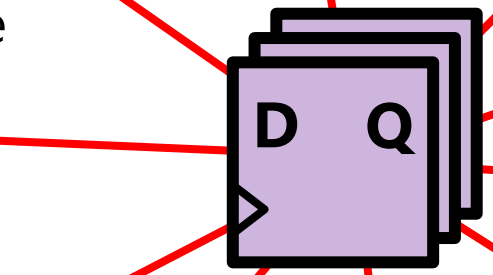
32/64 bits

why is this? well, what do you remember about the differences between RISC and CISC?

*or 32 *at a time*

Tug-of-war

- Register file design is constrained by many competing factors



compilers love lots of identical registers!…but there are diminishing returns.

ISA says instructions have 2 operands and 1 destination

fast L1 cache? not as many regs needed

…except for this one instruction that has 2 destinations.

multi-issue CPU: need to read 4 regs and write 2

humans like intuitive assembly language!

more registers means more silicon…

with *lots* of registers, function calls are faster!

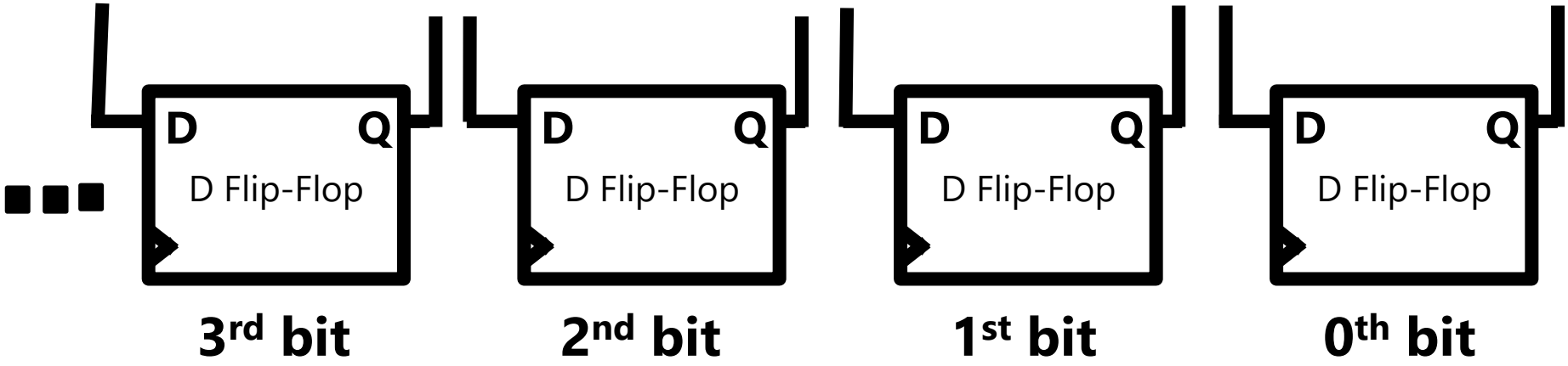
…but context switches are slower.

A word of advice

- You will see many imperfect designs in your life
- But in problem-solving, **perfection isn't always the goal**
  - everyone has to work within the constraints they're given
- and if *everyone* does something the same way…
  - there are probably problems/constraints you don't know about
  - **don't waste your time reinventing the wheel.**
    - find out *why* it's done that way first.
- When it comes to register files, 32 registers is just a nice number
  - not too many, not too few, a nice middle-ground
- also don't be a judgmental ass about someone else's design because one, it's shitty, and two, they know more about *why* it was designed that way, so you're just being presumptuous

Soooo registers… How do we create a register?

- If we create a 32-bit register out of D Flip-Flops:



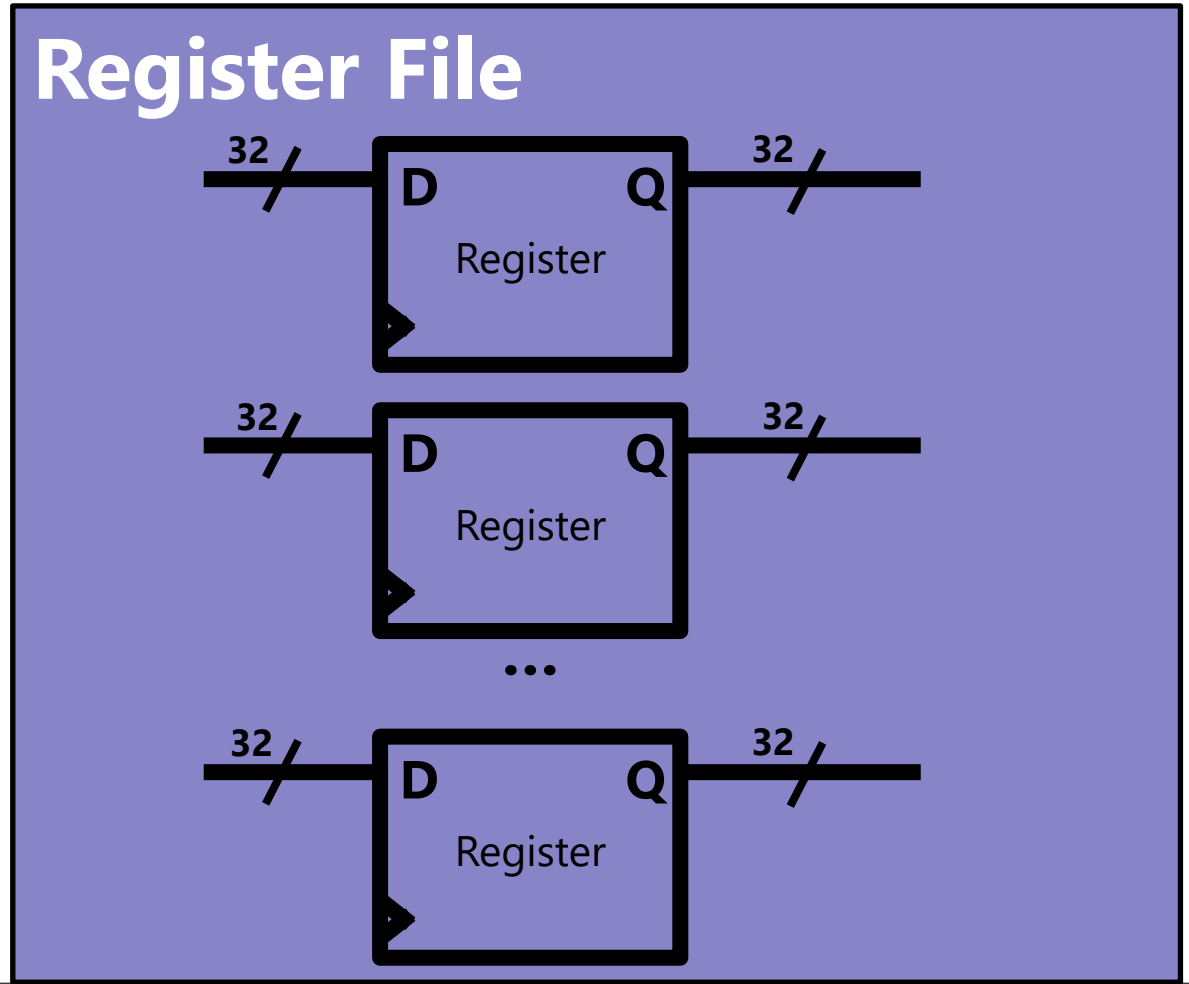
We abstract away to this:



Thankfully, so does Logisim!

Combined into…

- Then, we can combine many of those together:

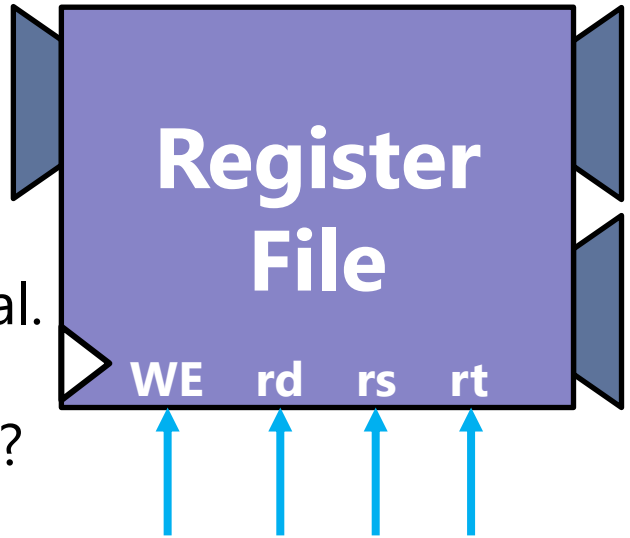


The MIPS register file

- In the instruction **`add t0, t1, t2`**, how many registers are **read?**
  - how many are **written?**
  - how many *different* registers are accessed?

there's one **input** or **write port**

it needs a clock signal. what other control signals does it need?



there are two **output** or **read** ports

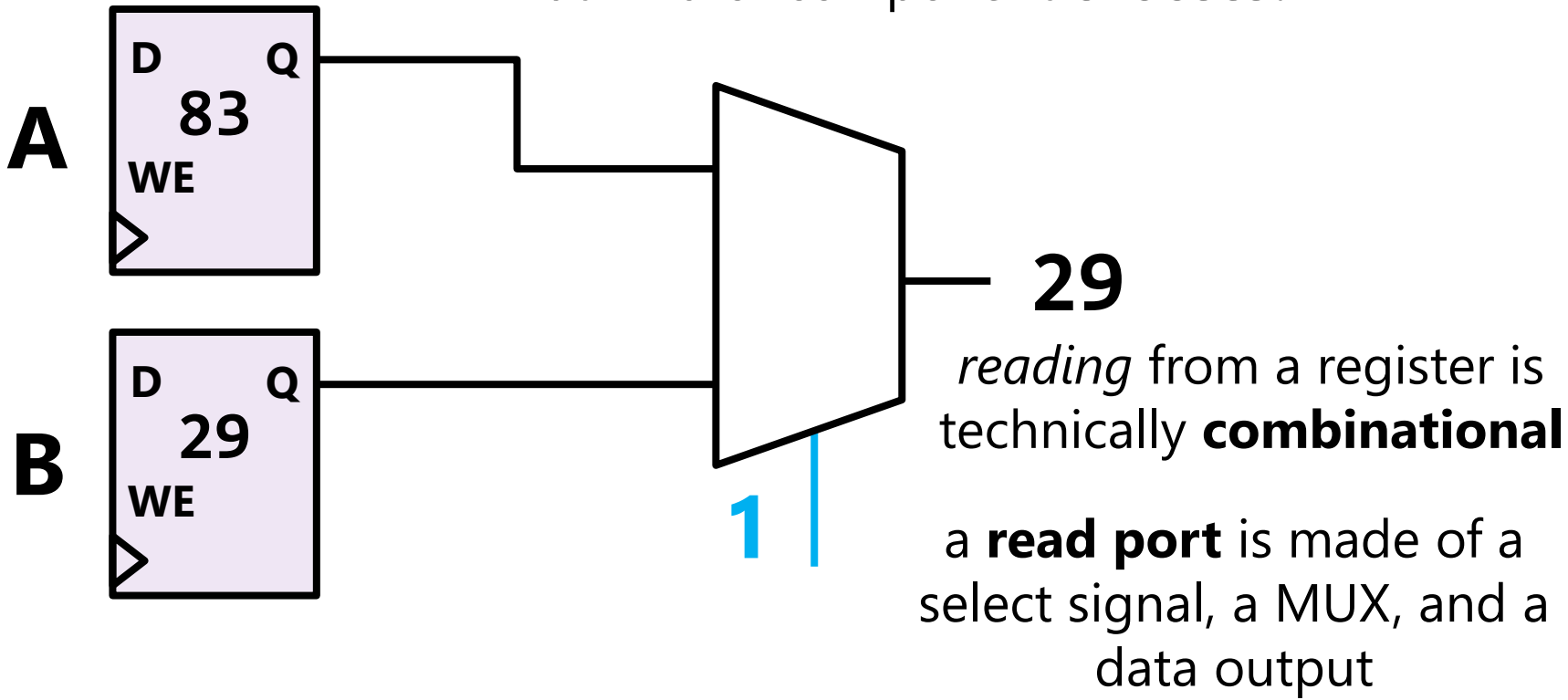
each port can read a **different** register

how about a write enable?

and inputs to select the registers?

Reading from one register

- You have two registers, and you want to **choose one to read**

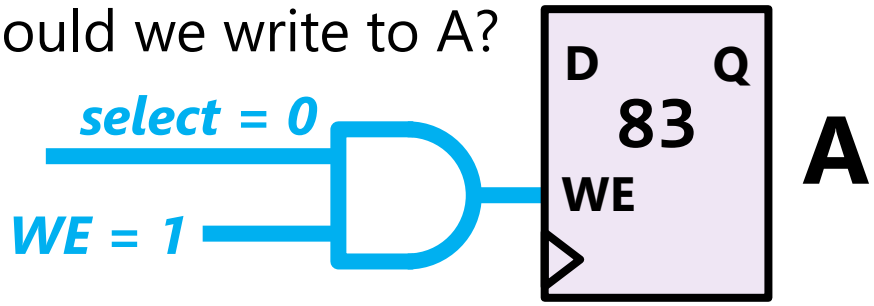


Writing

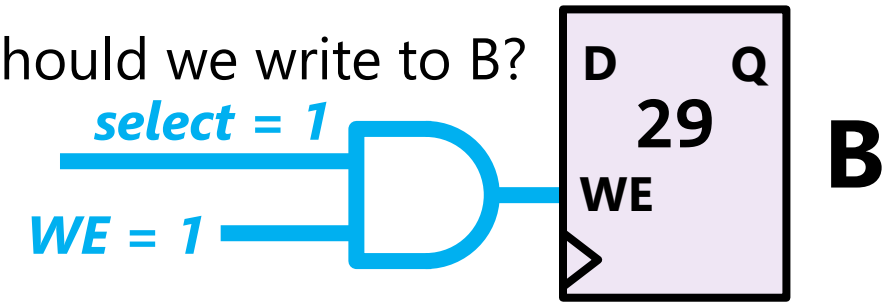
- For the **write port,** we only want to write to *one* register at a time
- We'll have a select signal again…

Do we ALWAYS write to a register?
How about in
`beq A, 3, top`?

when should we write to A?

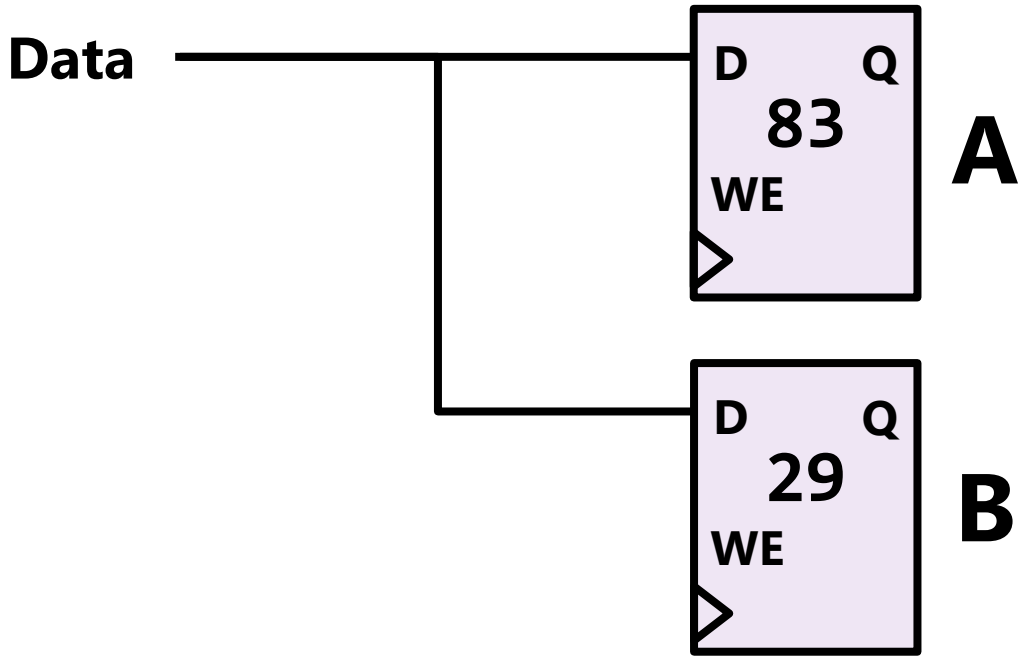


when should we write to B?



Close the door

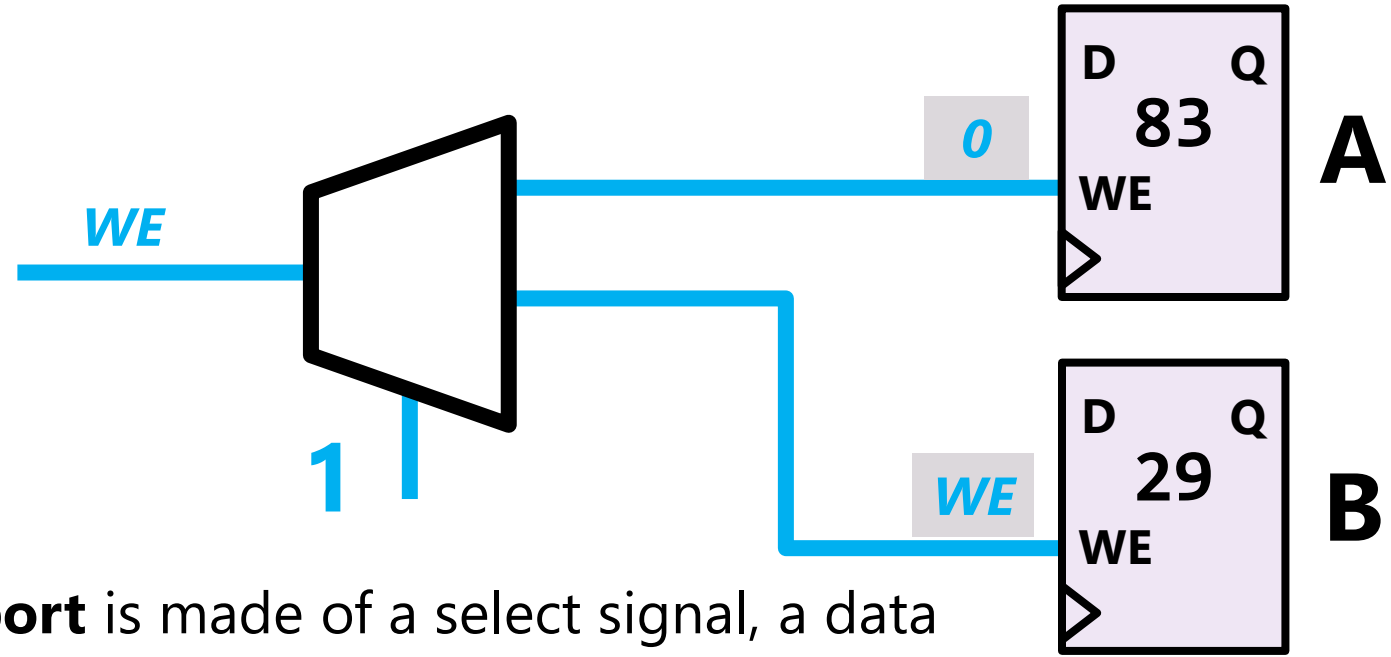
- when a register's write enable is 0, what happens to the data?
- so we can hook up the data input *to all registers at once*.



only the register with
WE=1 will store the data

Chekhov's Gun

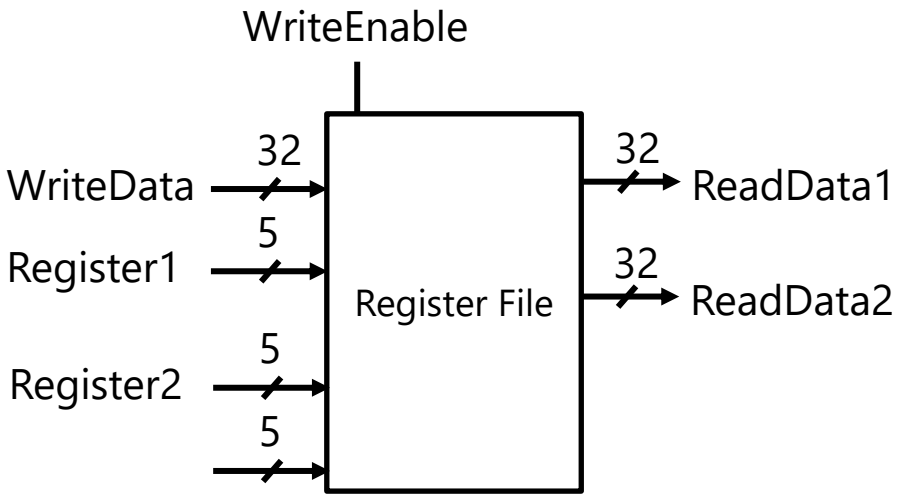
- there's a component we haven't seen in a while which *only sends an input value to* ***one*** *of its outputs (demux)*



a **write port** is made of a select signal, a data input, a write enable, and *some kinda logic* to send the write enable to **one** register

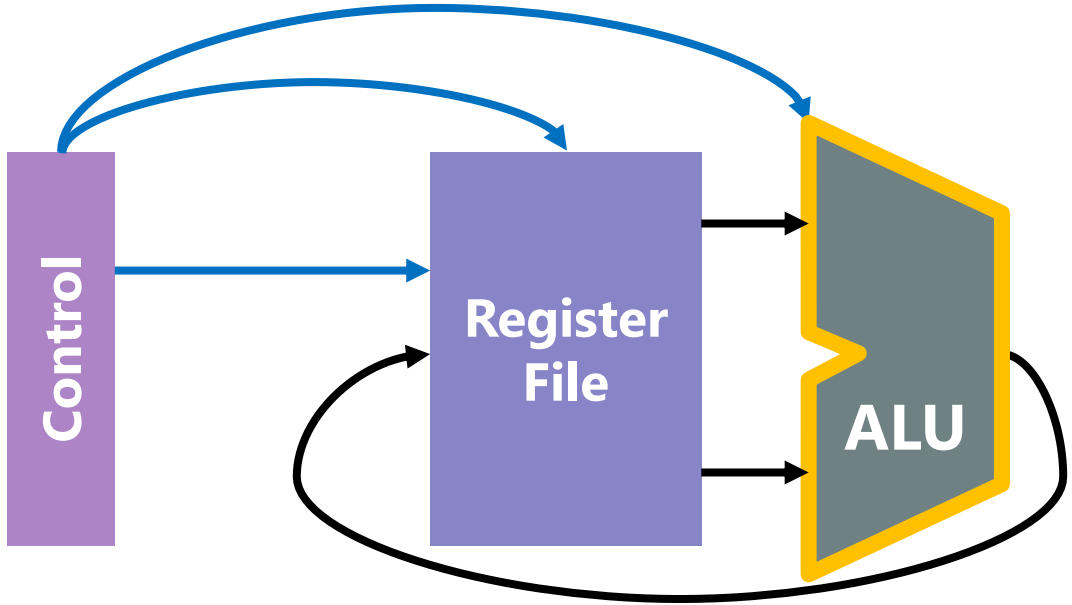
The Register File

- And then, we can abstract our subcircuit to the following:
  - This presumes we have 32 registers which are 32-bits in size
  - (like MIPS!)



Diving in

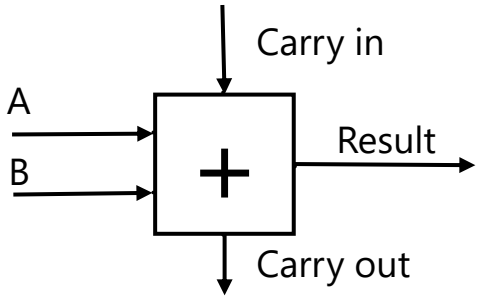
- We have a complete register file!
- Now… let's look more closely at building an ALU.



Building Out a Basic ALU

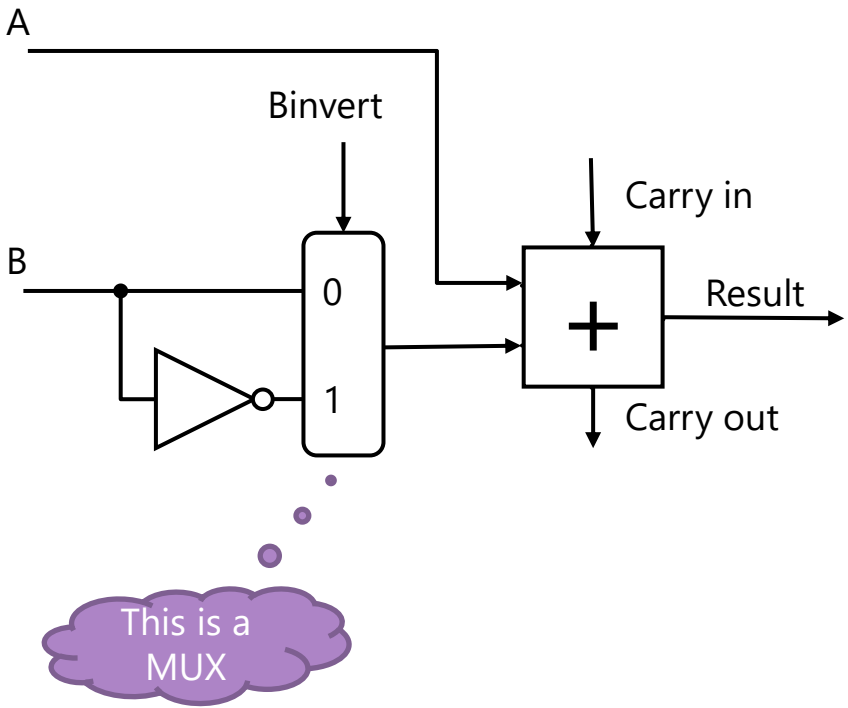
Starting small, the one-bit adder

- Who remembers how to use an adder to subtract?

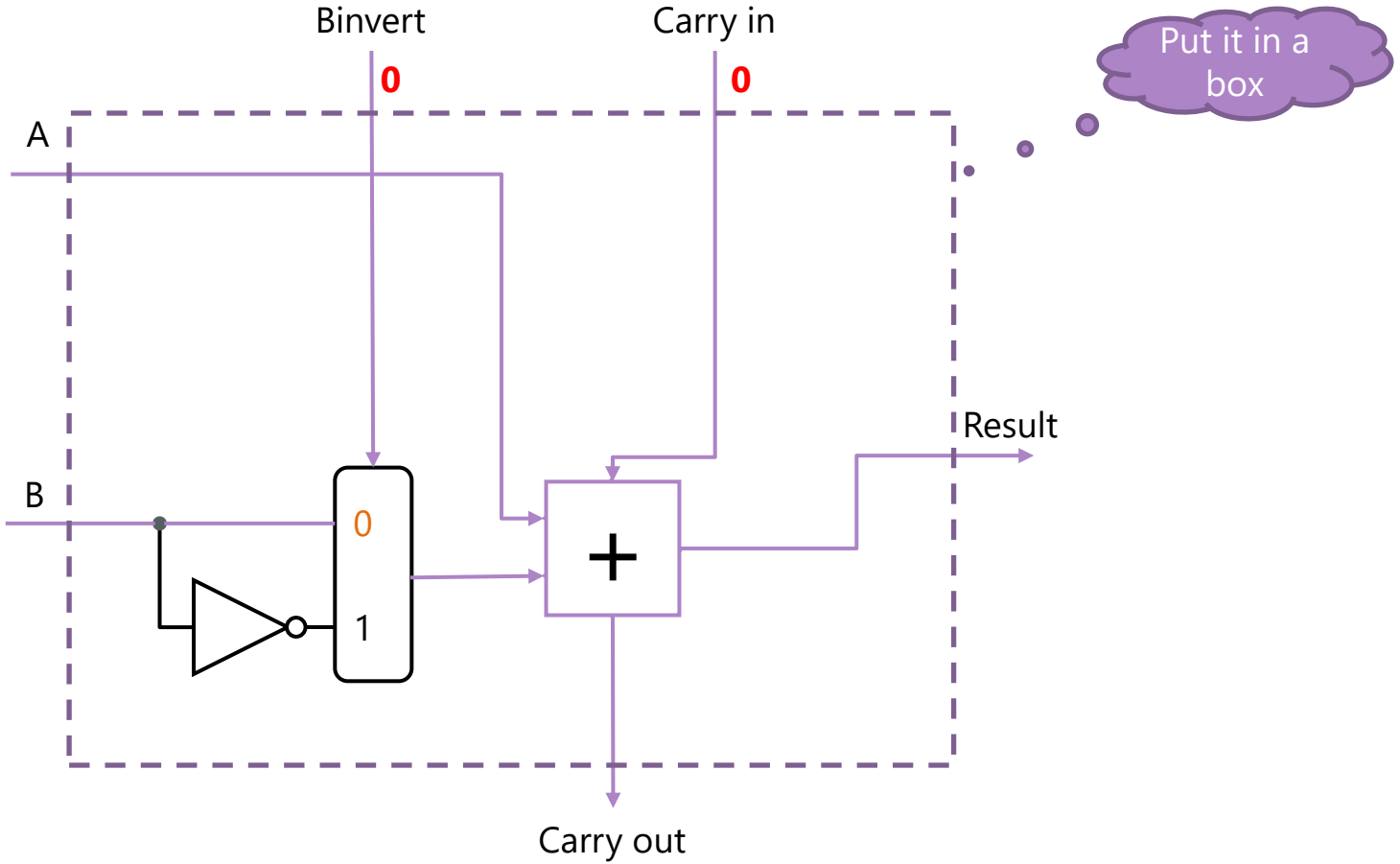


Starting small, the one-bit adder

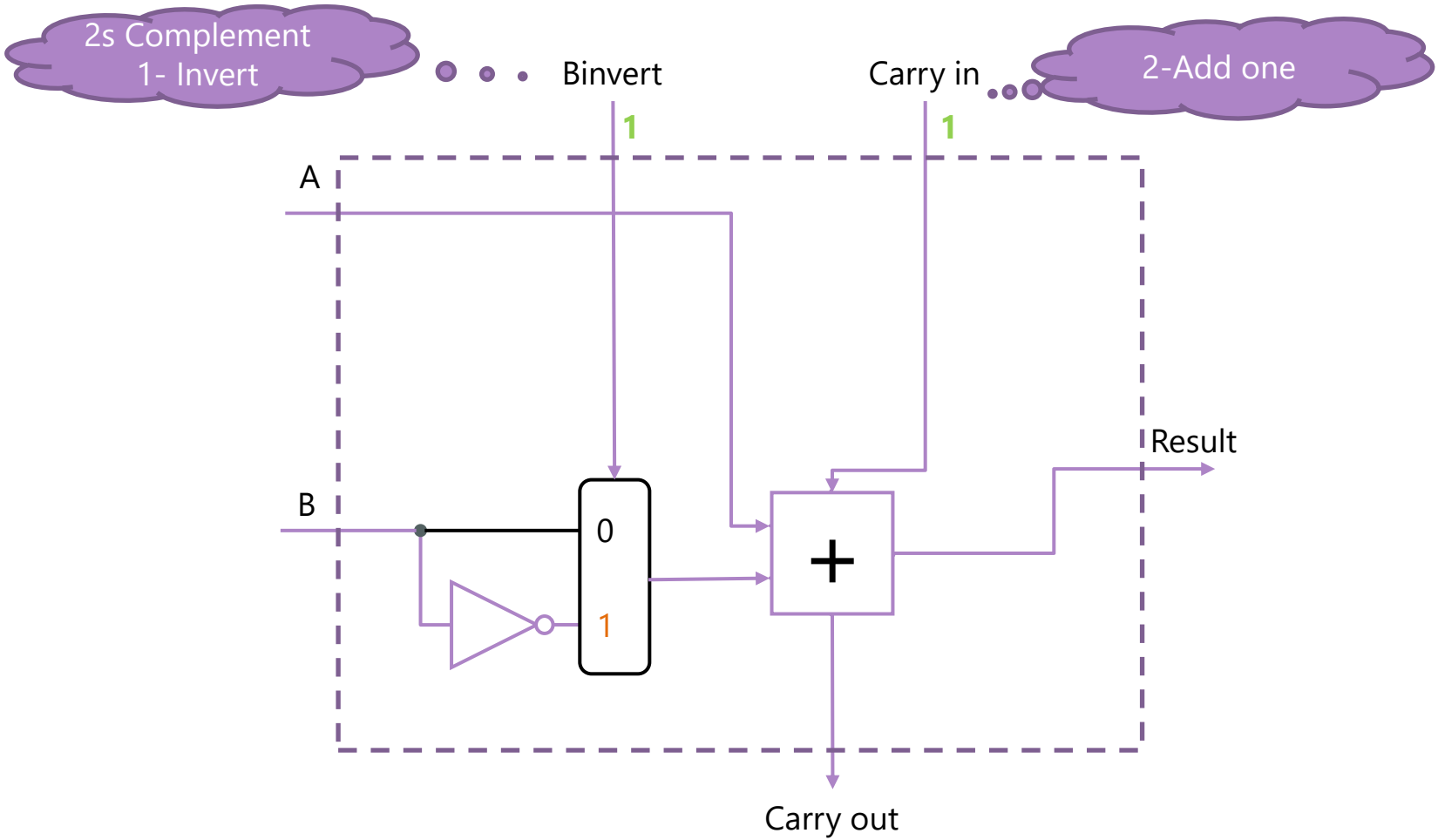
- Here is a simple ALU. It can Add A and B together.
  - There are a few **control signals** leading into it and several outputs.
  - Consider how this ALU subcircuit, as it is, can perform "A – B"



A basic 1-bit ALU – Addition



A basic 1-bit ALU – Subtraction



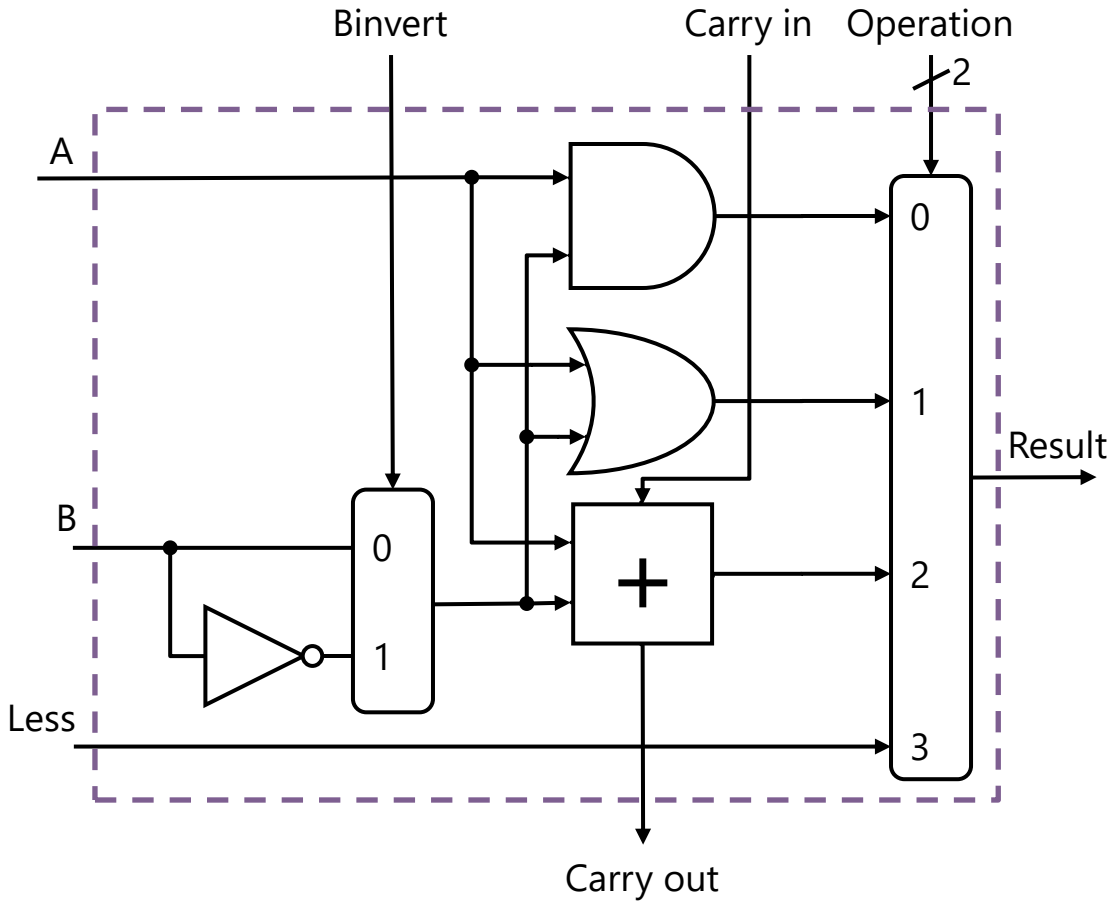
A basic 1-bit ALU – Expanding the adder

ALU – Arithmetic and Logic Unit

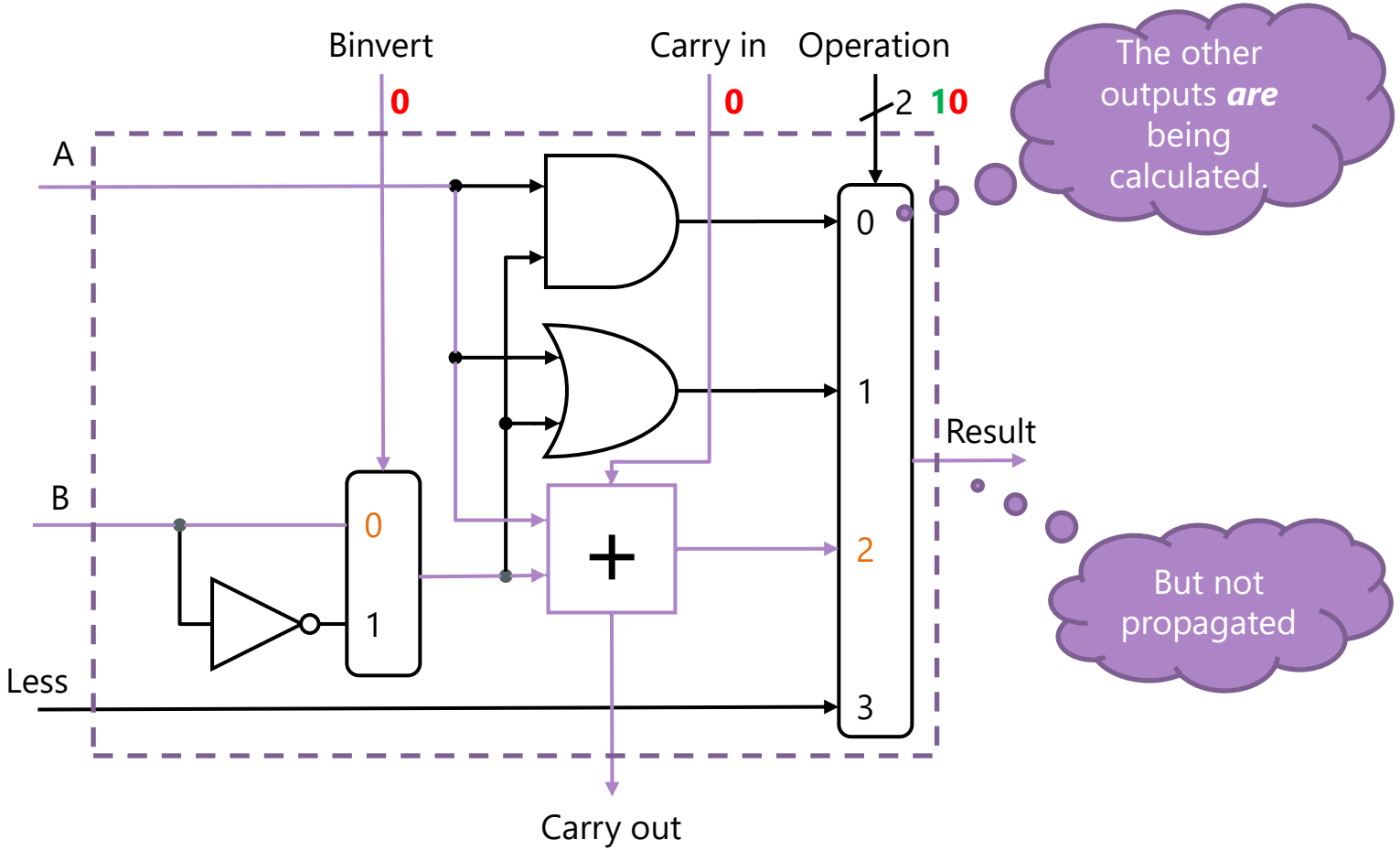
- This ALU can perform the arithmetic operations add, and subtract.
- And the logic operations AND, OR, and NOT
- Operation is selected by the signal Operation:

  (2-bits)

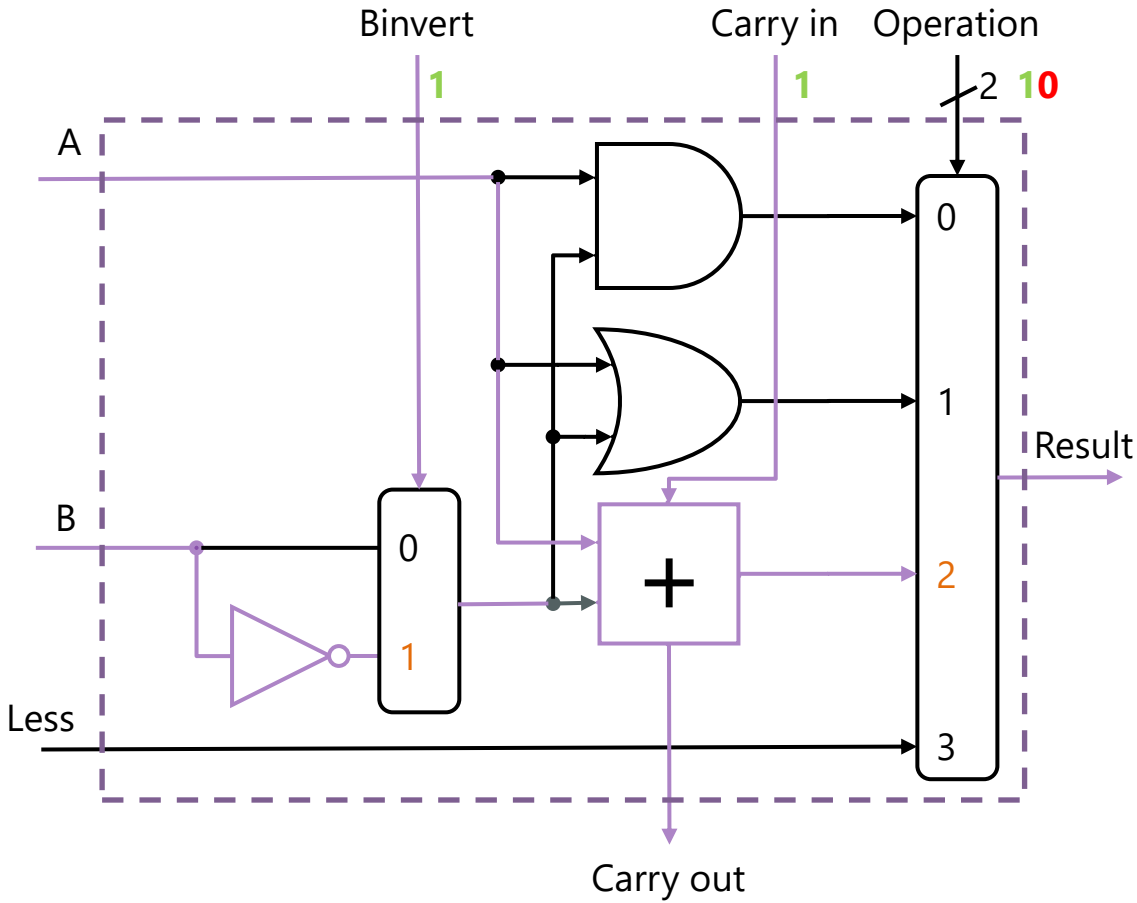
  00 – AND; 01 – OR;
  10 – ADD; 11 – SLT



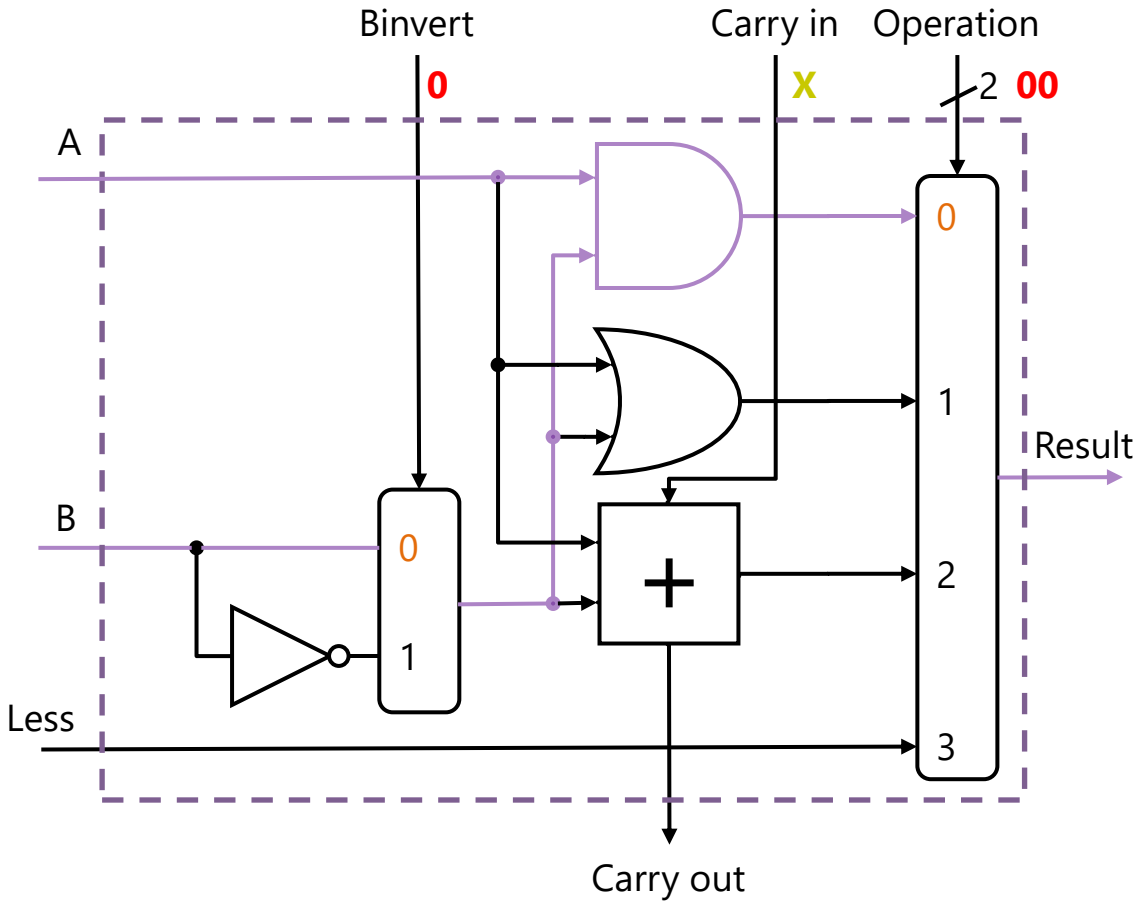
A basic 1-bit ALU – Addition



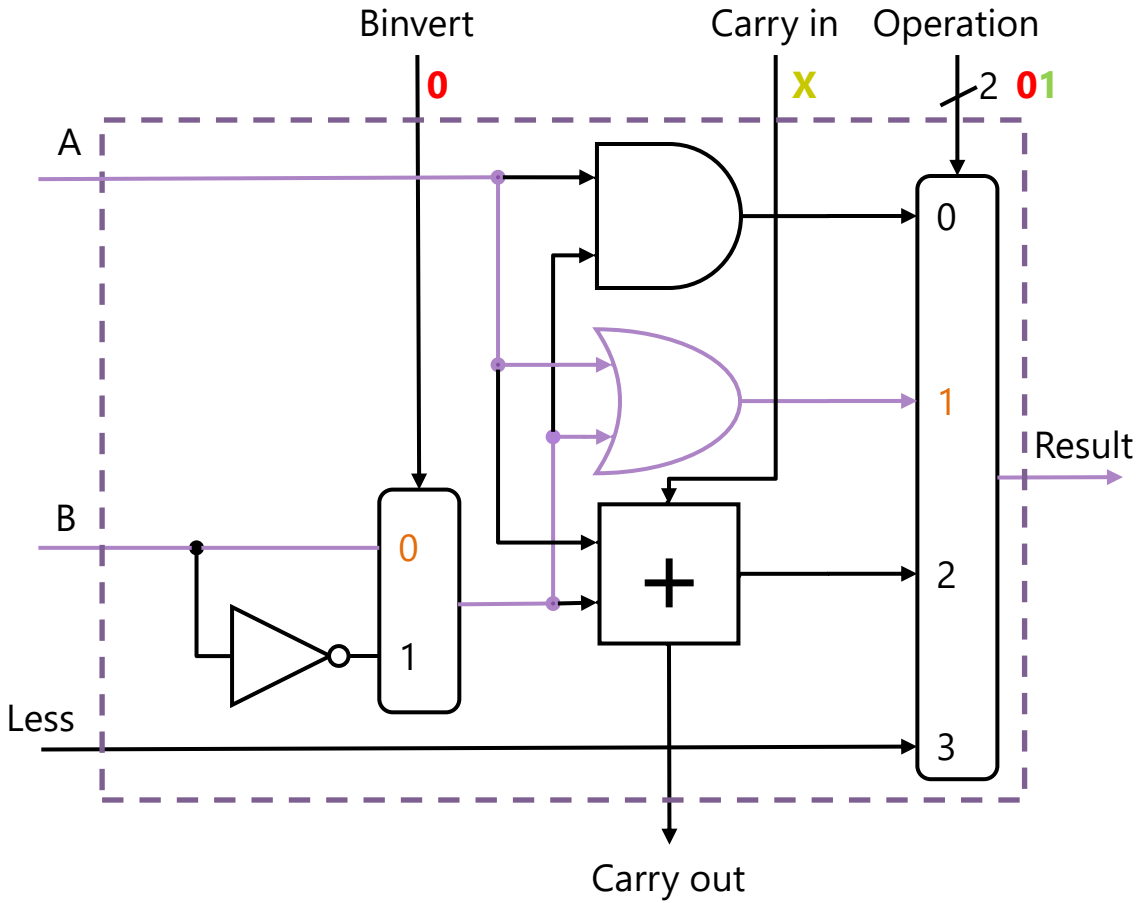
A basic 1-bit ALU – Subtraction



A basic 1-bit ALU – AND



A basic 1-bit ALU – OR



A basic 1-bit ALU – NAND and NOR?

Remember Boolean algebra?

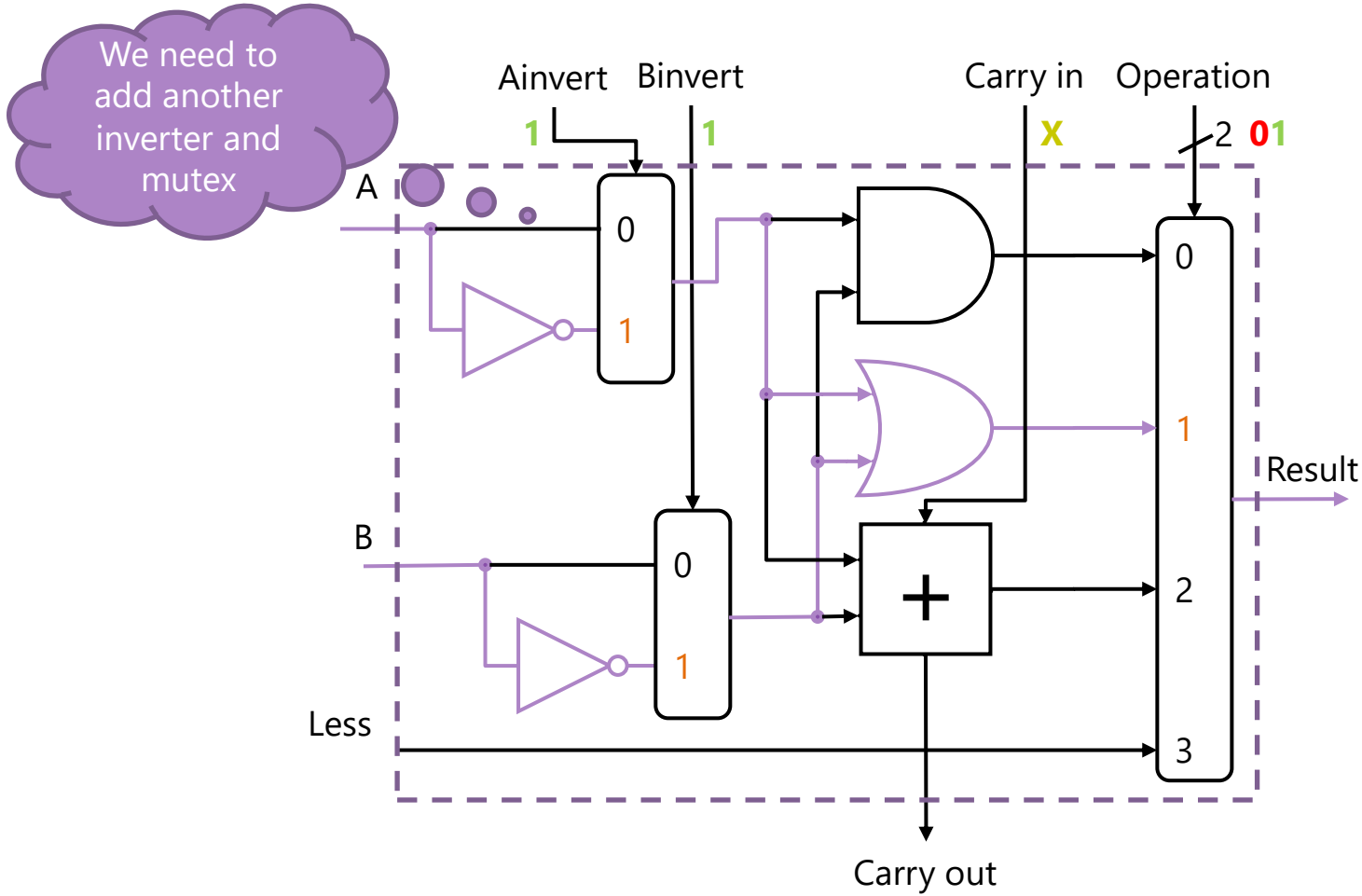
$$\overline{A + B} = \overline{A}.\overline{B}$$

A	B	$\overline{A + B}$	$\overline{A}$	$\overline{B}$	$\overline{A}.\overline{B}$
0	0	1	1	1	1
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	0

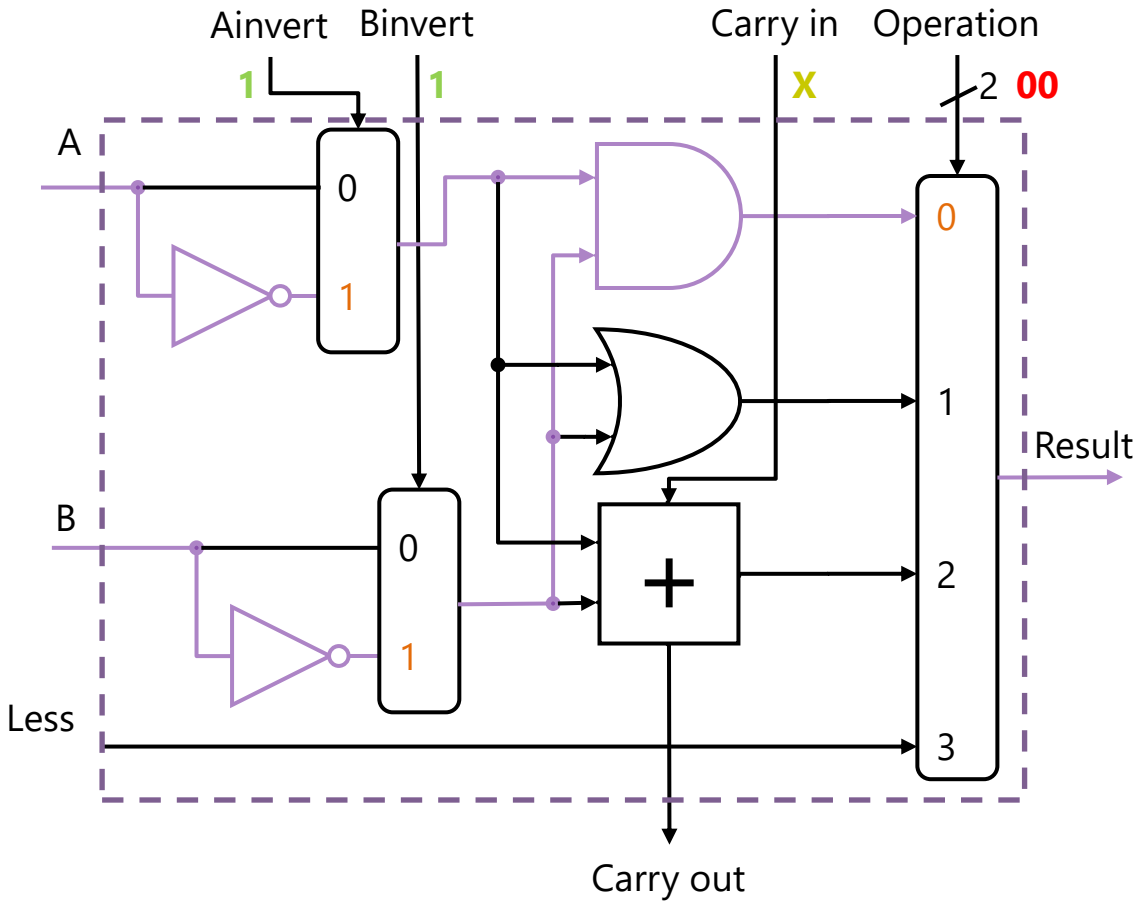
$$\overline{AB} = \overline{A} + \overline{B}$$

A	B	$\overline{AB}$	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	1	1	1	1
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0

A basic 1-bit ALU – NAND

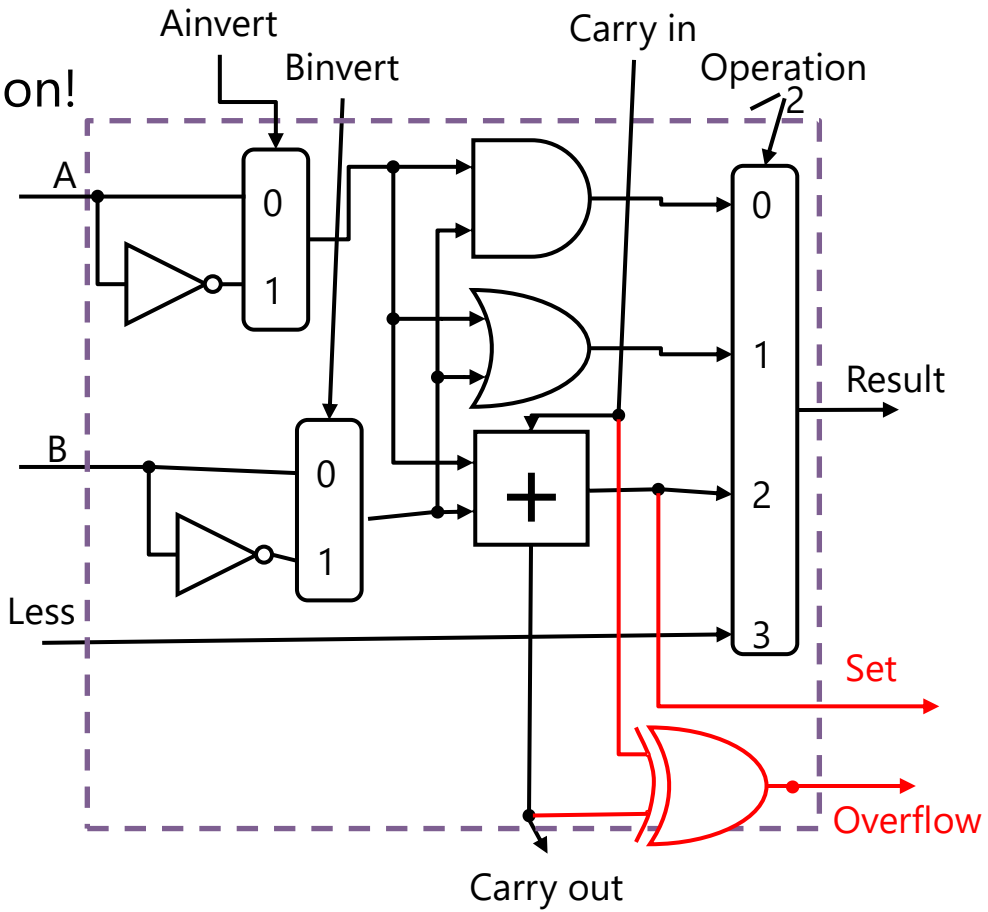


A basic 1-bit ALU – NOR



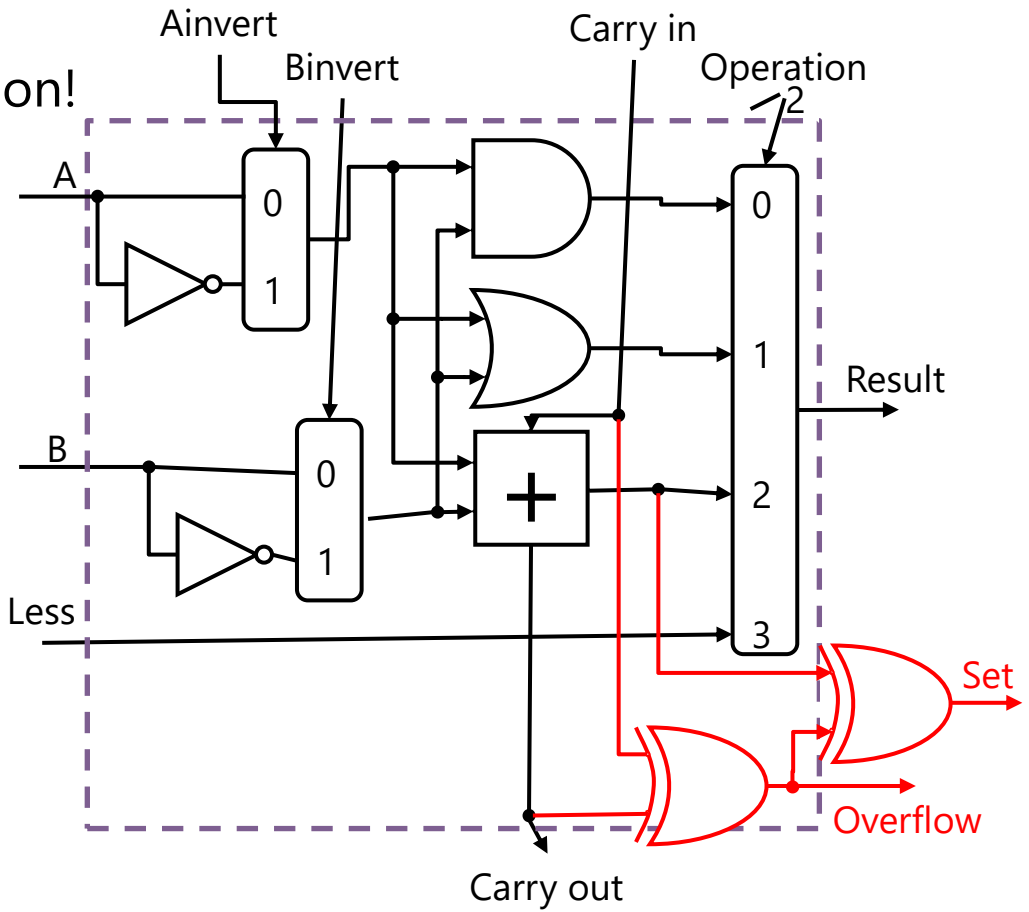
A basic 1-bit ALU, with overflow detection

- This ALU can detect overflow
- Also allows to perform the SLT operation!
- Remember the **slt** instruction?
  blt t0, t1, label
  - It's equivalent to
    slt at, t0, t1
    bnq at, zero, label
- SLT – Set if Less Then
  - "Set" = 1 if a<b
  - "Set" = 0 if a>=b



A basic 1-bit ALU, with overflow detection

- This ALU can detect overflow
- Also allows to perform the SLT operation!
- Remember the **slt** instruction?
  blt t0, t1, label
  - It's equivalent to
    slt at, t0, t1
    bnq at, zero, label
- SLT – Set if Less Then
  - "Set" = 1 if a<b
  - "Set" = 0 if a>=b
- What is the propagation delay?

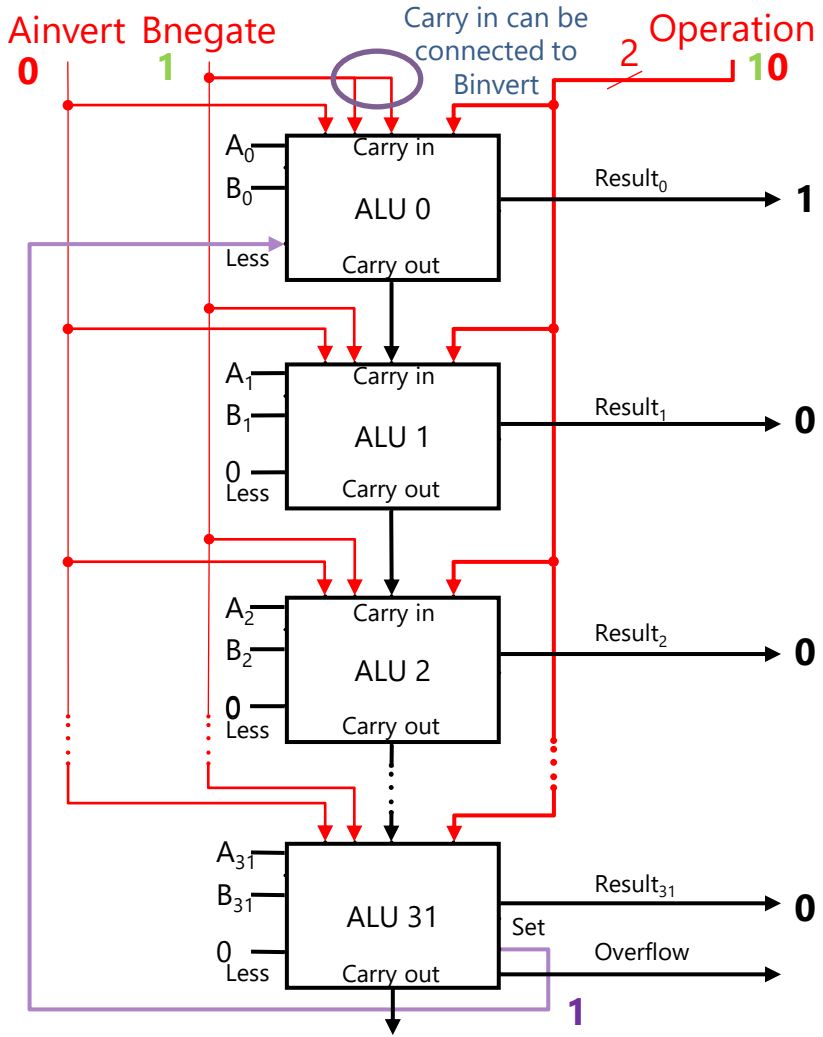


Assume: Not: 2ns, Mux: 6ns,
Adder: 10ns, AND/OR/XOR: 4ns

(26ns)

Building it up!

- Combine multiple 1-bit ALUs
- We can combine the *Binvert* and *Carry in* signals
  - They are used simultaneously for subtractions
  - Otherwise, we don't care about the *Carry in*

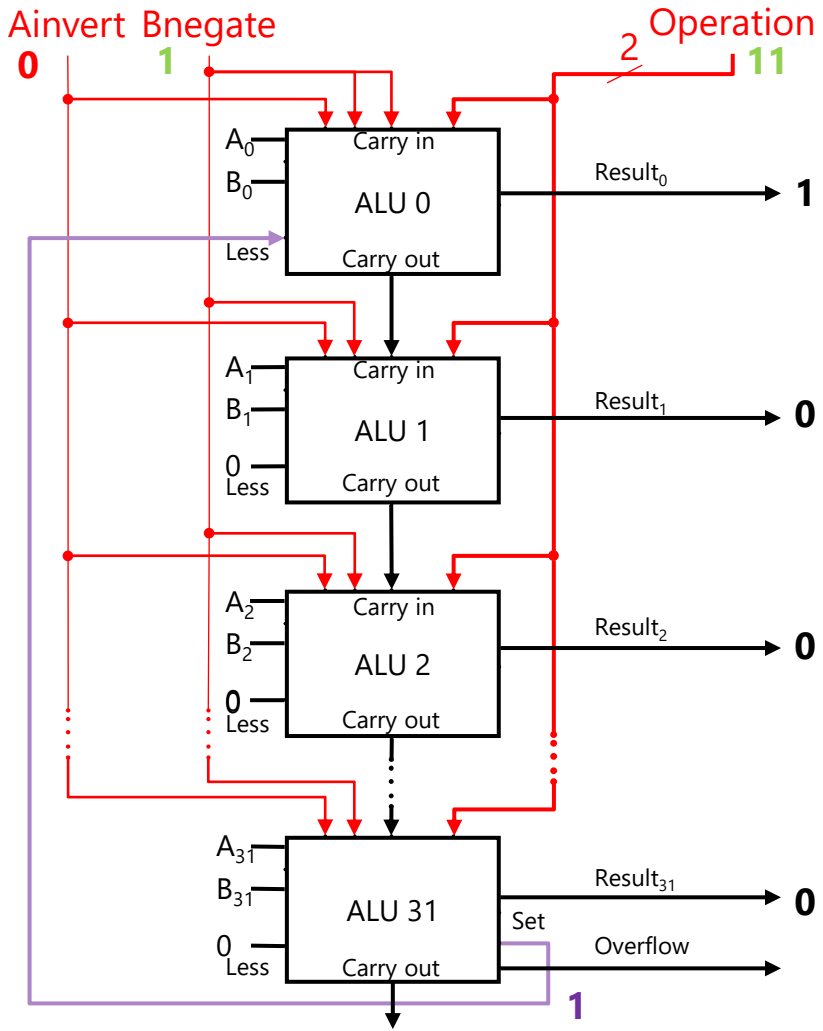


Implementing SLT

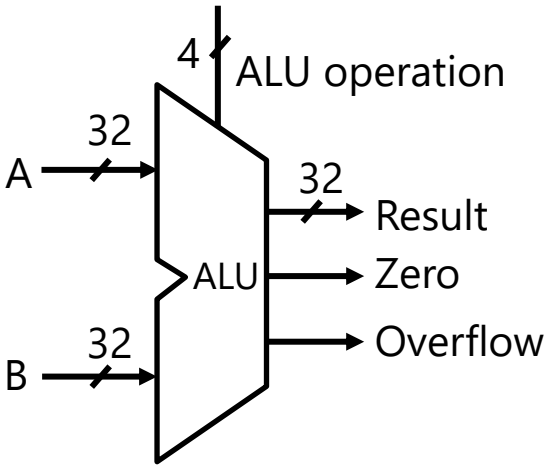
- SLT uses subtraction
  slt at, t0, t1
  t0<t1: t0-t1<0
  - Set is 1

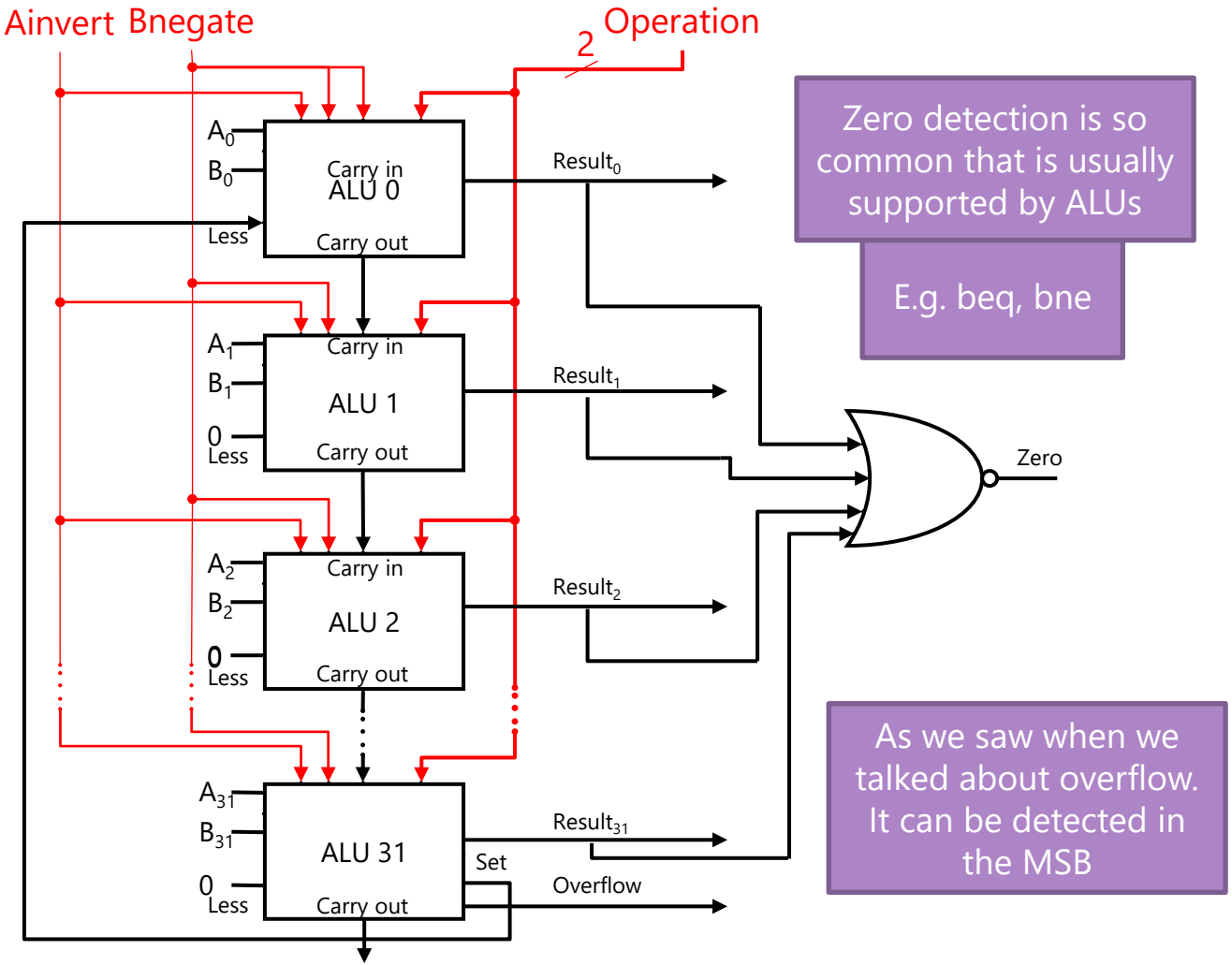
- Note how ***Set*** is connected to ALU0's ***Less*** input

- Could we use $Result_{31}$ instead?
  - No, note how the output is **0**, not **1**



A 32-bit ALU





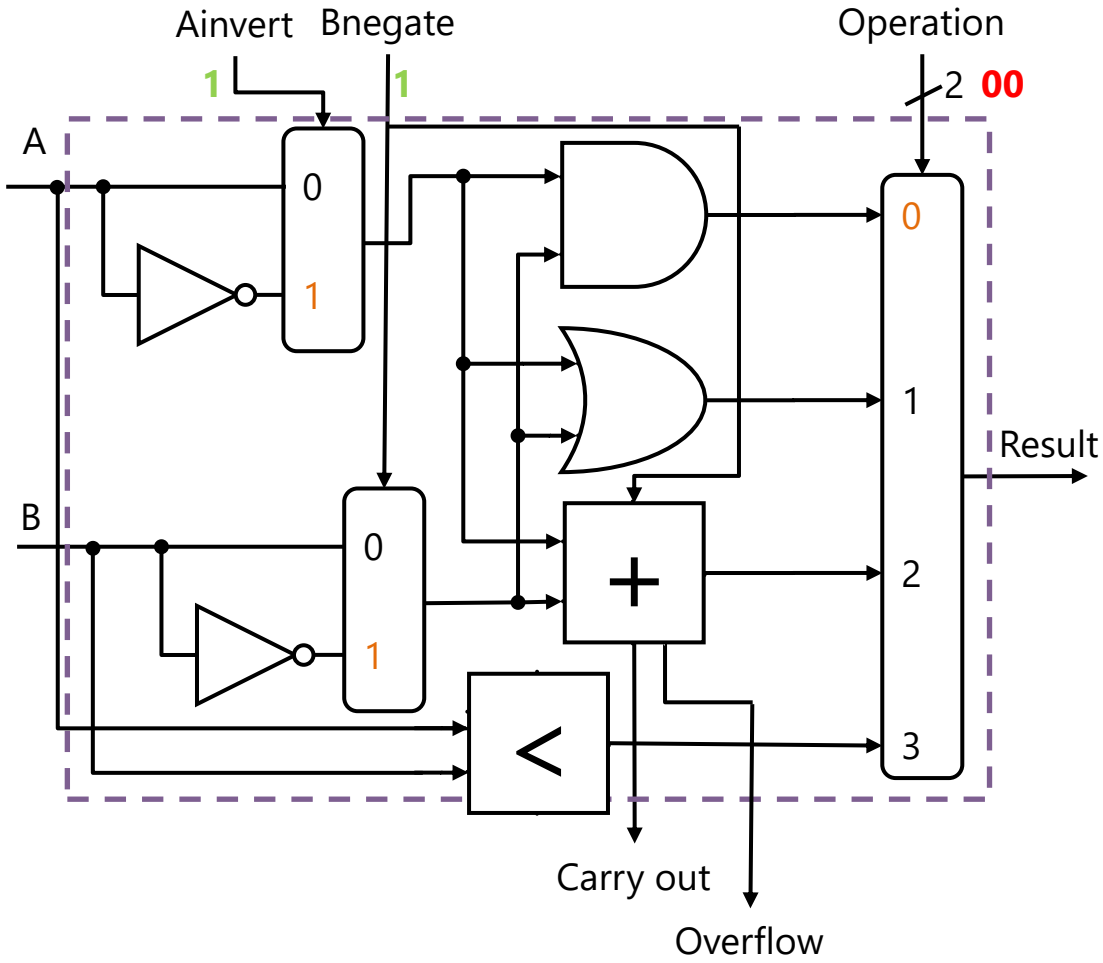
Zero detection is so common that is usually supported by ALUs

E.g. beq, bne

As we saw when we talked about overflow. It can be detected in the MSB

A basic 32-bit ALU?

- ALUs are many times in the real world built as multiple 1-bit ALUs.
  - Called **bit-slicing**
- However, we can happily live in our ideal world for a bit longer!
- We can build it much like the 1-bit model, but just tell Logisim to make the components' "Data Size" 32-bits.
  - i.e., for your project.
- Whew!



Zooming Out Again

- Now, how to we wire up the Register File with the ALU??
  - How do we know which registers to use?
  - How do we pull in instructions?
- Tune in next time for…
  - Control and Datapath

